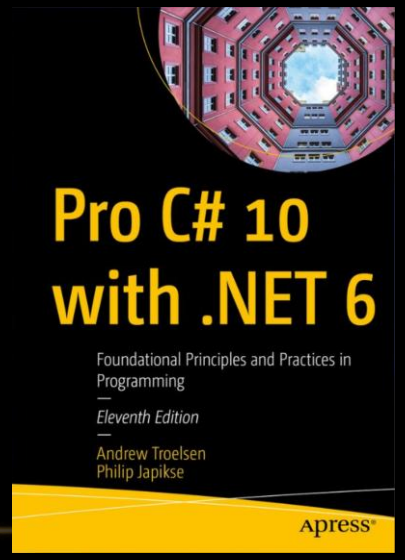
ASP.NET CORE RAZOR PAGES

Philip Japikse (@skimedic)
skimedic@outlook.com
CTO, Author, Teacher
Microsoft MVP, ASPInsider, PST, PSM II, PSD

All slides copyright Philip Japikse http://www.skimedic.com

Phil.About()

- CTO/Chief Architect, Pintas & Mullins
- Author: Apress.com (http://bit.ly/apressbooks)
- Professional Scrum Trainer (PSF, PSD)
- Speaker: http://www.skimedic.com/blog/page/Abstracts.aspx
- Microsoft MVP, ASPInsider, PST, PSM II, PSD
- Founder, Agile Conferences, Inc.
  - http://www.cincydeliver.org
- President, Cincinnati .NET User's Group



All slides copyright Philip Japikse http://www.skimedic.com

SUPPORT LIFECYCLES

All slides copyright Philip Japikse http://www.skimedic.com

.NET CORE SUPPORT LIFECYCLES

- Long Term Support (LTS)
  - Upgraded with patches – typically security
  - Supported for three years after GA release
  - In Support:
    - 8.0 LTS (support until 11/10/26)
    - 9.0 STS (support until 11/10/26)
    - 10.0 LTS (support until 11/14/28)
- ~~Current~~ Standard Term Support (STS)
  - Minor releases
  - Upgraded more rapidly
  - Supported for ~~6 months after the next release (LTS or STS)~~
    - 2 Years after release
  - Out of Support:
    - 5.0 support ended 05/10/22
    - 6.0 support ended 11/12/24
    - 7.0 support ended 05/14/24

https://www.microsoft.com/net/core/support

All slides copyright Philip Japikse http://www.skimedic.com

MIGRATING FROM FRAMEWORK TO ASP.NET CORE

All slides copyright Philip Japikse http://www.skimedic.com

ASP.NET (MVC/WEBAPI) -> ASP.NET CORE

- I Recommend:
  - File -> New project
  - Clipboard inheritance for controllers, views, and services

- Can try: https://dotnet.microsoft.com/platform/upgrade-assistant
  - Last I tried it, it didn’t handle EF Core upgrade very well

All slides copyright Philip Japikse http://www.skimedic.com

WCF -> ASP.NET CORE

- WCF is now supported through CoreWCF
  - Open-source port of WCF (.NET F/W) to .NET

All slides copyright Philip Japikse http://www.skimedic.com

WEB FORMS

- YARP provides an upgrade path
  - .NET application fronts the Webforms application
  - Anything not implemented in .NET is handed off to your Webforms
  - Allows for migration over time

- NOTE: I have not used this in real life

All slides copyright Philip Japikse http://www.skimedic.com

ASP.NET CORE FUNDAMENTALS

All slides copyright Philip Japikse http://www.skimedic.com

ASP.NET CORE

- ASP.NET Core rebuilt on top of .NET Core
- Single framework for web, services, and microservices
  - WebApi + MVC + Razor Pages + Blazor = ASP.NET Core
- Cross-platform
  - Not tied to IIS or Windows
- Takes advantage of .NET Core performance
  - Includes a high performance web server (Kestrel) built on LibUV

All slides copyright Philip Japikse http://www.skimedic.com

ASP.NET CORE FEATURES

- Pluggable Middleware
  - Routing, authentication, static files, etc.
- Full Dependency Injection integration
- Environmental Awareness
- Simplified and Improved Configuration System
- Tag Helpers
- View Components
- Plus more!

All slides copyright Philip Japikse http://www.skimedic.com

NOTABLE UPDATES

- ASP.NET Core 6
  - Blazor Wasm
  - CSS Isolation
  - Minimal APIs
  - “Hot” Reload
  - C# 10
- ASP.NET Core 7
  - New Middleware
  - SignalR client results
  - C# 11
- ASP.ENT Core 8
  - Blazor Web App (Server-side + Wasm)
  - Improved Authentication/Authorization
    - Blazor Identity UI
    - Identity API endpoints for JS clients
    - And more!
  - SignalR seamless reconnect
  - Keyed Services
  - C# 12

All slides copyright Philip Japikse http://www.skimedic.com

NOTABLE UPDATES IN ASP.NET CORE 9

- Developer exception page improvements
  - Routing information, better formatting, improved readability
- Fixing 503's during app recycle
- Data protection key deletion
- Middleware support of Keyed DI
- Trust HTTPS dev cert on Linux

All slides copyright Philip Japikse http://www.skimedic.com

NOTABLE UPDATES IN ASP.NET CORE 9 WEB APPS

- Static Asset Delivery Optimization (MVC, Razor Pages, Blazor Server)
  - Caching, Compression, + more
  - NOTE: Does not minify or bundle
- Version updates in the default template for:
  - Bootstrap (5.3.3)
  - jQuery (3.7.1)
  - jQuery Validation (1.21.0)

All slides copyright Philip Japikse http://www.skimedic.com

NOTABLE UPDATES IN ASP.NET CORE 9 FOR SERVICES

- Minimal APIs - ProblemDetails on route groups
- OpenAPI package – Microsoft.AspNetCore.OpenApi
  - Nice progress, not quite there (no XML Comments support, no UI, versioning is still rough)
- Support for Pushed Authorization Requires (PAR)
- OIDC and Oauth parameter customization

All slides copyright Philip Japikse http://www.skimedic.com

NOTABLE UPDATES IN ASP.NET CORE 10

- Authentication and Authorization
  - Auth metrics
  - Avoid cookie login redirects for known end points
- Performance
  - Automatic eviction from the memory pool
  - Managed memory pools
  - Plus many more…
- OpenAPI
  - OpenAPI 3.1 support
  - Response Description
  - XML doc comment support
  - Plus a ton more!
- Minimal APIs
  - Validation Support
  - Support for Server-Sent Events (SSE)
- Support for JSONPatch
- Better local URL detection
- Blazor
  - New and updated security
    - OICD, Entra ID, Windows Auth
  - Lots more

All slides copyright Philip Japikse http://www.skimedic.com

CREATING THE WEB HOST

All slides copyright Philip Japikse http://www.skimedic.com

ASP.NET CORE APPS ARE CONSOLE APPS

- Web host is created in top-level statements
  - Responsible for app startup and lifetime management
- At a minimum, it configures a server and request processing pipeline
- Everything is opt-in!

```
ıġεAyyǿoʃcóoţşĆºoǿğġÿ%ε°oǿğġÿ%○%ıġεAyyǿoʃcóoţşĆºoǿğġÿ<cÿłÿ/{|
:≔Čţşkol°ÿcóoţş%ţk%ÿġÿMoʃġÿ%cşğ%kÿcšġΠţÿőÿ%<Πcÿ%Čţşkol°ÿġÙġÿoʃġÿ<///
ıġεAyyǿoʃcóoţş%cyy%○%ε°oǿğġÿ<Ćºoǿğ/{|
:≔Čţşkol°ÿcóoţş%ţk%ǿňġ%ЮŪŪŢ%yoyġǿoşġ%<Πcÿ%Čţşkol°ÿġ<///
cyy<Úºş/{|
```

All slides copyright Philip Japikse http://www.skimedic.com

DEPENDENCY INJECTION

All slides copyright Philip Japikse http://www.skimedic.com

ADDING CUSTOM DEPENDENCIES TO DI CONTAINER

- Configured in the Program.cs file
  - After creating the app builder and before building the app (ConfigureServices)
  - In .NET 8+, can be keyed

- Used to configure any services needed by the application
  - Transient – created each time they are requested
  - Scoped – created once per request
  - Singleton – created once

- Can plug in different IOC framework

All slides copyright Philip Japikse http://www.skimedic.com

BUILT-IN DEPENDENCY INJECTION

- Services are accessed through:
  - Constructor injection
  - Method injection
  - [MVC/Razor] View injection
- Can also retrieve services through:
  - ApplicationServices (for non-controller classes)
  - HttpContext.RequestServices (for controllers)
  - Injection is the preferred mechanism

All slides copyright Philip Japikse http://www.skimedic.com

INJECTING SERVICES INTO CONTROLLERS/PAGES/VIEWS

- Dependencies are added in Program.cs (ConfigureServices)

```
builder.Services.AddScoped<ICarRepo, CarRepo>();
```

- Class constructors by the DI container

```
y°εøo∫%Čÿġcóġỏṭğġø<ĬČcÿÚġyṭ%∫cÿÚġyṭ/
```

- Page Handlers need [FromServices]

```
public void OnGet([FromServices] ICarRepo carRepo)
```

- Views need the @inject directive

```
@inject <type> <name>
@inject ICarRepo carRepo
```

All slides copyright Philip Japikse http://www.skimedic.com

INJECTING KEYED SERVICES (8+)

- Dependencies are added in Program.cs (ConfigureServices)

```
builder.Services.AddKeyedScoped<ISimpleService, SimpleService>("key");
```

- Class constructors

```
y°εøo∫%Ùošyøġ̀Ùġÿ̀Mo∫ġỌ̃ṭğġø<.Ĝÿṭ̣šMġΦġğ̀Ùġÿ̀Mo∫ġÿ̃<//őġØ//#=ÌÙošyøġ̀Ùġÿ̀Mo∫ġ%ÿ̃ġÿ̀Mo∫ġø/
```

- PageHandlers

```
public void OnGet(.Ĝÿṭ̣šMġΦġğ̀Ùġÿ̀Mo∫ġÿ̃<//őġØ//#=ÌÙošyøġ̀Ùġÿ̀Mo∫ġ%ÿ̃ġÿ̀Mo∫ġ)
```

- Views need the @inject directive for IServiceProvider

```
@inject IServiceProvider serviceProvider
@{
   var service = serviceProvider
                 .GetKeyedService<ISimpleService>(nameof(SimpleServiceOne));
}
```

All slides copyright Philip Japikse http://www.skimedic.com

VALIDATE CONTROLLER SERVICES, VIEW COMPONENTS, TAG HELPERS ON RUN

- Add this to Program.cs (ConfigureServices)

```
builder.Services.AddRazorPages()
   .AddControllersAsServices()
   .AddViewComponentsAsServices()
   .AddTagHelpersAsServices();
```

```
builder.Host.UseDefaultServiceProvider(o =>
{
   o.ValidateOnBuild = true;
   o.ValidateScopes = true;
});
```

All slides copyright Philip Japikse http://www.skimedic.com

ENVIRONMENTAL AWARENESS APPLICATION CONFIGURATION

All slides copyright Philip Japikse http://www.skimedic.com

ENVIRONMENTAL AWARENESS

- ASP.NET Core uses ASPNETCORE_ENVIRONMENT variable
  - Development, Staging, and Production are built-in environments

- Environment is used throughout ASP.NET Core
  - Configuration
  - C# Execution paths
  - HTML/CSS/JS

- Simplifies deployment and testing

All slides copyright Philip Japikse http://www.skimedic.com

APPLICATION CONFIGURATION

- Applications are configured using:
  - Default = Simple JSON
    - Other file types also supported
  - Command line arguments, Environment variables, In memory .NET objects, Encrypted user store, Custom providers

- Values are set in the order received - last in wins
  - appsettings.json
  - Environment determines which additional file to load
    - appsettings.<environment>.json

All slides copyright Philip Japikse http://www.skimedic.com

THE OPTIONS PATTERN

All slides copyright Philip Japikse http://www.skimedic.com

THE OPTIONS PATTERN

- Add custom classes to the services collection
- Classes are reconstituted from the application settings
- Injected using one of the IOptions<T> interfaces
  - Typically using IOptionsSnapshot or IOptionsMonitor

All slides copyright Philip Japikse http://www.skimedic.com

USING THE OPTIONS PATTERN

- Add the option pattern to the DI container

```
services.Configure<CustomSettings>(Configuration.GetSection("CustomSettings"));
```

- Inject the class into a Page constructor/handler method:

```
public IndexModel(IOptionsMonitor<DealerInfo> dealerOptionsMonitor)
   => Entity = dealerOptionsMonitor.CurrentValue;
public void OnGet(IOptionsSnapshot<DealerInfo> dealerOptionsSnapshot)
   => Entity = dealerOptionsSnapshot.Value
```

All slides copyright Philip Japikse http://www.skimedic.com

MODEL BINDING AND VALIDATION

All slides copyright Philip Japikse http://www.skimedic.com

MODEL BINDING

- Route data and query string values are used only for simple types.
- Implicit model binding of complex types uses BIND_PROPERTY
  - Explicit model binding is also available

- Errors in data type conversion are added to ModelState

All slides copyright Philip Japikse http://www.skimedic.com

MODEL BINDING

- Sources for data (in order):
  - Form fields
  - The request body (API services)
  - Route data
  - Query string parameters
  - Uploaded files

All slides copyright Philip Japikse http://www.skimedic.com

MODEL BINDING EXPLICIT SOURCES

- Can explicitly specify the source:
  - [FromQuery] - query string.
  - [FromRoute] - route data.
  - [FromForm] - posted form fields.
  - [FromBody] - request body.
  - [FromHeader] - HTTP headers.

- Can also explicitly call for model binding with code

All slides copyright Philip Japikse http://www.skimedic.com

VALIDATION

- Data annotations offer built-in client and server-side validation
  - Custom validation attributes are also available

- Implicit validation occurs with model binding
  - Explicit validation is also available

- Errors in validation are added to ModelState

All slides copyright Philip Japikse http://www.skimedic.com

CUSTOM VALIDATION

All slides copyright Philip Japikse http://www.skimedic.com

CUSTOM VALIDATION ATTRIBUTES

- Derive from ValidationAttribute for server-side validation
  - Override the IsValid method
  - ValidationContext provides access to the entire model

- Implement IClientModelValidator for client-side validation
  - AddValidation methods adds data- attributes to rendered elements
  - Ties into JS validation (e.g. Jquery validations)

All slides copyright Philip Japikse http://www.skimedic.com

LOGGING

All slides copyright Philip Japikse http://www.skimedic.com

LOGGING IS BUILT INTO FRAMEWORK

- LoggerFactory is part of the framework
- Extensible using 3rd party loggers (e.g. SeriLog)

All slides copyright Philip Japikse http://www.skimedic.com

RAZOR PAGES

All slides copyright Philip Japikse http://www.skimedic.com

RAZOR PAGES

- Built on top of ASP.NET Core MVC framework
  - Still uses MVC style layout, partials, and templates

- Razor Page:
  - Markup (View) e.g. Index.cshtml
  - Code-behind (PageModel) e.g. Index.cshtml.cs
  - Routing defined by folder structure and page directive

All slides copyright Philip Japikse http://www.skimedic.com

PAGEMODEL

- Base class for pages.
- Contains helper methods for:
  - Returning response messages
    - No body: NoContent (204), Ok (200), BadRequest (400)
    - With body: OkObjectResult(car) (200)
  - Model Binding and Validation
  - Redirecting requests

All slides copyright Philip Japikse http://www.skimedic.com

RAZOR PAGES

- Page Handlers respond to Get and Post requests
  - OnGet[Async]() or OnPost[Async]()
  - Note: Names *can* be changed
- Antiforgery is on by default for all Post requests
- ViewBag largely replaced with properties from the code behind

All slides copyright Philip Japikse http://www.skimedic.com

ROUTING

All slides copyright Philip Japikse http://www.skimedic.com

ROUTING

- Routing is bi-directional
  - Determines which Controller/Action or PageHandler to execute
  - Generates URLs within the application

- Get/Post requests are handled by the HTTP Verb-specific page handlers
  - OnGet[Async]/OnPost[Async]

All slides copyright Philip Japikse http://www.skimedic.com

RAZOR PAGE ROUTING

- Route is based on file structure and name
- Route parameters are added to the @page directive
- Reserved tokens use “{}”

All slides copyright Philip Japikse http://www.skimedic.com

THE POST-REDIRECT-GET PATTERN (PRG)

- When a Post action completes, redirect the user to a Get action
- E.g. Create -> Details, Delete -> Index
- Prevents user double posting

All slides copyright Philip Japikse http://www.skimedic.com

VIEWS AND LAYOUTS

All slides copyright Philip Japikse http://www.skimedic.com

RAZOR PAGE VIEWS

- Rendered from a route to a Page
- Razor code mixed with markup
- Tag Helpers inject/update markup to prevent context switching

- Located in Pages, Pages\PathName, or Pages\Shared

All slides copyright Philip Japikse http://www.skimedic.com

PARTIAL VIEWS

- Don’t use a layout
- Render from an action method/page handler using PartialView()
- Render from another view using partial tag helper

All slides copyright Philip Japikse http://www.skimedic.com

LAYOUTS

- _ViewStart.cshtml sets the default for views
  - Can be configured per view

- Layout
  - RenderBody renders the view
  - Sections add more control (required || optional)
    - RenderSection/IgnoreSection

- Partial Views don't use a layout

All slides copyright Philip Japikse http://www.skimedic.com

TEMPLATES

- Render automatically if named after a type and located
- Rendered on demand by passing in the name

- DisplayTemplates (DisplayFor/DisplayForModel)
  - Located in ..\DisplayTemplates folder

- EditorTemplates (EditFor/EditForModel)
  - Located ..\EditorTemplates

All slides copyright Philip Japikse http://www.skimedic.com

<ScopedCssEnabled>false</ScopedCssEnabled>

CSS ISOLATION

- Views, Layouts, and Pages can have their own CSS files
- All get bundled when published or in development into {ProjectName}.styles.css
  - Can force bundling by calling UseStaticWebAssets

```
if (!builder.Environment.IsDevelopment())
{
    builder.WebHost.UseStaticWebAssets();
}
```

- Can disable in the project file:
  - <ScopedCssEnabled>false</ScopedCssEnabled>
  - <DisableScopedCssBundling>true</DisableScopedCssBundling>

All slides copyright Philip Japikse http://www.skimedic.com

EU GENERAL DATA PROTECTION REGULATION (GDPR)

All slides copyright Philip Japikse http://www.skimedic.com

GDPR SUPPORT

- Extension points and stubbed markup for privacy and cookie use policy.
- Cookie consent feature asks (and tracks) consent
  - Without consent, non-essential cookies aren't sent to the browser.
  - Cookies can be marked as essential.
  - TempData and Session cookies aren't functional when tracking is disabled.
- The Identity manage page provides a link to download and delete user data.

- ASP.NET Core 3.1 templates removed starter code

All slides copyright Philip Japikse http://www.skimedic.com

TAG HELPERS

All slides copyright Philip Japikse http://www.skimedic.com

TAG HELPERS

- Enable server-side code to participate in rendering HTML elements in Razor views
- Reduces the transition between code and markup
  - Tag Helpers Attach to HTML elements
  - HTML Helpers are invoked as methods
- Fully supported with IntelliSense

- Can also create custom tag helpers

All slides copyright Philip Japikse http://www.skimedic.com

THE FORM TAG HELPER

- Supported tags (must include at least one):
  - asp-area || asp-controller || asp-action
  - asp-antiforgery (true by default)*
  - asp-route-<parametername> (e.g. asp-route-id="1")
  - asp-all-route-data – uses a dictionary for the route data
  - asp-route (for named routes)
- Area, Controller, Action are defaulted to current route
- Equivalent functionality to @Html.BeginForm/EndForm

All slides copyright Philip Japikse http://www.skimedic.com

FORM FIELD TAG HELPERS

- Include Input, TextArea, Select, and Label tags
- Model property is selected with "asp-for“
  - Provides strong typing with model properties
- Generate Id and Name values based on model property
- Add HTML5 validation attributes based on model definition

All slides copyright Philip Japikse http://www.skimedic.com

FORM FIELD TAG HELPERS (CONTINUED)

- Input (@Html.TextBoxFor or @Html.EditorFor)
  - Adds HTML type based on .NET data type or data annotation
    - E.g. Bool => type="checkbox", [EmailAddress] => type="email"

- Label (@Html.LabelFor)
  - Generates label caption and "for" attribute

- Select (@Html.DropDownListFor or @Html.ListBoxFor)
  - Generates option elements based on "asp-items" attribute

All slides copyright Philip Japikse http://www.skimedic.com

VALIDATION TAG HELPERS

- Validation Message (@Html.ValidationMessageFor)
  - Property selected with asp-validation-for
  - Generates data-valmsg-for attribute

- Validation Summary (@Html.ValidationSummary)
  - Enabled with asp-validation-summary

All slides copyright Philip Japikse http://www.skimedic.com

NON-FORM TAG HELPERS

- Anchor
- Environment
- Link/Script/Image
- Partial

All slides copyright Philip Japikse http://www.skimedic.com

THE ANCHOR TAG HELPER

- Supported tags:
  - asp-area || asp-controller || asp-action
  - asp-protocol (for http/https)
  - asp-route-<parametername> (e.g. asp-route-id="1")
  - asp-all-route-data – uses a dictionary for the route data
  - asp-route (for named routes)
  - asp-fragment
  - asp-hostname

All slides copyright Philip Japikse http://www.skimedic.com

THE ENVIRONMENT TAG HELPER

- Conditionally renders content based on the run-time environment
- The "names" attribute accepts one or more environment names
  - If HostingEnvironment.EnvironmentName matches, content is loaded

- Changed in 2.0:
  - Added the "include" and "exclude" attributes

All slides copyright Philip Japikse http://www.skimedic.com

THE LINK TAG HELPER

- The "asp-append-version" tag adds hash of file to URL
  - Resolves issue of files still cached when contents change
  - Adds ?v=<hash of file> to the url
- Href handling tags:
  - asp-href-include/exclude – globbed file list to include/exclude
  - asp-fallback-test-class – class used to test original source
  - asp-fallback-test-property – property used to test original source
  - asp-fallback-test-value – value used to test original source
  - asp-fallback-href – fall back file to use if primary file fails to load
  - asp-fallback-href-include || exclude – globbed file list to include/exclude in fall back

All slides copyright Philip Japikse http://www.skimedic.com

THE SCRIPT TAG HELPER

- The "asp-append-version" tag adds hash of file to URL
  - Resolves issue of files still cached when contents change
  - Adds ?v=<hash of file> to the url
- Href handling tags:
  - asp-src-include/exclude – globbed file list to include/exclude
  - asp-fallback-test – script method to test success of loading source
  - asp-fallback-src – fall back file to use if primary file fails to load
  - asp-fallback-src-include || exclude – globbed file list to include/exclude in fall back

All slides copyright Philip Japikse http://www.skimedic.com

THE IMAGE TAG HELPER

- The "asp-append-version" tag adds hash of file to URL
  - Resolves issue of files still cached when contents change
  - Adds ?v=<hash of file> to the url

All slides copyright Philip Japikse http://www.skimedic.com

THE PARTIAL TAG HELPER

- Used to invoke partial views asynchronously
  - Name attribute is required
  - For is the model passed into the view
  - Model used to create a new model instance
  - View-data passes a view data dictionary to the partial

- Used in place of:
  - @await Html.PartialAsync, @await Html.RenderPartialAsync
  - @Html.Partial, @Html.RenderPartial

All slides copyright Philip Japikse http://www.skimedic.com

THE CACHE TAG HELPER

- Provides a way to mark content as cached using the <cache> tag
- Supports absolute, timespan or sliding expiration
- Supports additional cache options:
  - vary-by-header – Cache is evicted when single or list of header values change
  - vary-by-query – Cache is evicted when single or list of query values change
  - vary-by-route – Cache is evicted when single or list of route parameters change
  - vary-by-cookie – Cache is evicted when single or list of cookies change
  - vary-by-user – Cache is evicted when context principal changes
  - vary-by – Allows for further customization of cache data and timeout

All slides copyright Philip Japikse http://www.skimedic.com

THE DISTRIBUTED CACHE TAG HELPER

- Inherits from Cache tag helper
  - All attributes for Cache tag helper are supported
- Supports SQL Server or Redis as a distributed cache

All slides copyright Philip Japikse http://www.skimedic.com

CUSTOM TAG HELPERS

All slides copyright Philip Japikse http://www.skimedic.com

CUSTOM TAG HELPERS

- Class inherits TagHelper
- Class name (minus TagHelper) becomes the element name
  - E.g. EmailTagHelper == <email></email>
- Public properties are added as lower kebob cased attributes
  - E.g. EmailName == email-name="foo@foo.com"
- Must opt in to use (usually in the _ViewImports.cshtml partial)
  - @addTagHelper *, AutoLot.MVC

All slides copyright Philip Japikse http://www.skimedic.com

VIEW COMPONENTS

All slides copyright Philip Japikse http://www.skimedic.com

VIEW COMPONENTS

- View Components combine server-side code with partial views
  - Used to render a chunk of the response
  - Used instead of ChildActions/PartialViews
- Common Uses:
  - Dynamically created menus
  - Login panel
  - Shopping cart

All slides copyright Philip Japikse http://www.skimedic.com

CREATE VIEWCOMPONENT CLASS

- Class derives from ViewComponent
  - Can name ending in ViewComponent or decorate with [ViewComponent]

- Implement Invoke/InvokeAsync and return IViewComponentResult
  - Any data needed for view passed into view as viewmodel
  - Returns a partial view

All slides copyright Philip Japikse http://www.skimedic.com

CREATE PARTIAL VIEW FOR VIEWCOMPONENT

- Create standard partial view
  - Default name is “default.cshtml"

- Must locate partial view in:

```
Views/<controller_name>/Components/<view_component_name>/<view_name>
Views/Shared/Components/<view_component_name>/<view_name>
Pages/Shared/Components/<View Component Name>/<View Name>
```

All slides copyright Philip Japikse http://www.skimedic.com

INVOKING VIEW COMPONENTS

- Must update _ViewImports.cshtml
  - @addTagHelper *, AutoLot.Web
- Invoke from a view (or layout) using lower-kebob-casing:
  - <vc:view-component-name model="@Model">
- Invoke from a controller action method:
  - return ViewComponent("<name>“, <anonymous type with parameters>);

All slides copyright Philip Japikse http://www.skimedic.com

MANAGING CLIENT SIDE LIBRARIES

All slides copyright Philip Japikse http://www.skimedic.com

LIBRARY MANAGER

- Built into VS 2022/2026
- Available as a global dotnet tool
- Libraries are managed in libman.json
  - Sources include CDNJS, unpkg, file system

All slides copyright Philip Japikse http://www.skimedic.com

BUNDLING AND MINIFICATION

All slides copyright Philip Japikse http://www.skimedic.com

add app.UseWebOptimizer() to the Configure method anywhere before app.UseStaticFiles

ASP.NET CORE WEB OPTIMIZER

- Provides run time bundling/minification
- Files are added to cache and has cache busting built-in
- Package is LigerShark.WebOptimizer.Core
- Configure in Program.cs (ConfigureServices)
- Add to pipeline in Program.cs (Configure)
  - Must be called before app.UseStaticFiles()

All slides copyright Philip Japikse http://www.skimedic.com

IHTTPCLIENTFACTORY

All slides copyright Philip Japikse http://www.skimedic.com

IHTTPCLIENTFACTORY

- ASP.NET Core 2.1 includes a new IHttpClientFactory service that makes it easier to configure and consume instances of HttpClient in apps.
- The factory:
  - Makes registering of instances of HttpClient per named client more intuitive.
  - Implements a Polly handler that allows Polly policies to be used for Retry, CircuitBreakers, etc.
  - Handles pooling and lifetime management of HttpClient

All slides copyright Philip Japikse http://www.skimedic.com

AREAS

All slides copyright Philip Japikse http://www.skimedic.com

AREAS

- Partitioned “mini” MVC sites within an application
- Used for parts of the application that should be partitioned from the main app (e.g. admin functions)
- Can move main ViewStart and ViewImports to the project-level to work with all views

All slides copyright Philip Japikse http://www.skimedic.com

Contact Me

skimedic@outlook.com
www.skimedic.com/blog
www.twitter.com/skimedic

http://bit.ly/apressbooks

Questions?

Thank You!

All slides copyright Philip Japikse http://www.skimedic.com